

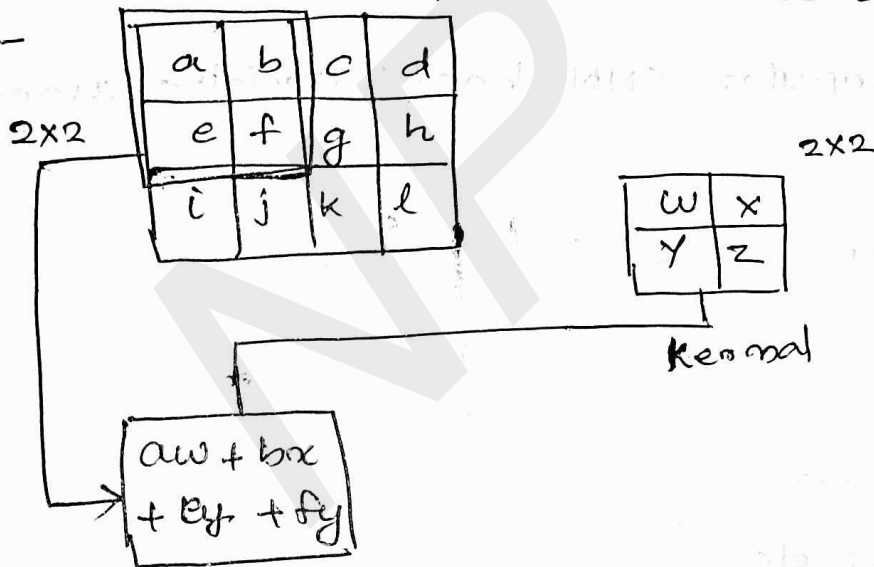
MODULE-3

CNN

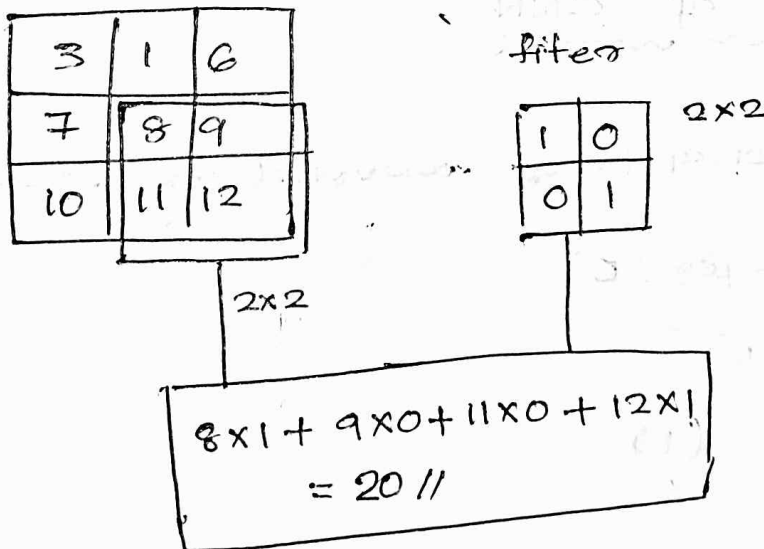
* CNN is a specialized Neural network used for processing Grid like topology data

example $\rightarrow$ Image.

* CNN uses convolution operation on the data
example -



eg:-



CNN is defined as the networks that use convolution operation in atleast one layer.

Applications of CNN

- * It is used to extract features in the data
- * It can be used for classification task.
- * It can be used for segmentation task.

CNN Based Architectures

Different popular CNN based architectures are

- Lenet
- Alexnet
- ZFnet
- VGG
- Googlenet.
- Resnet etc.

Basic Structure of CNN

A CNN consists of convolution layers (C)

- Convolution layer (C)
- ReLU Layer (R)
- Pooling Layer (P)

(i) Convolution Layer

- * Convolution layer performs convolution operation on the input data

(ii) ReLU Layer

ReLU layer. apply activation function

(ii) Pooling Layer

Pooling Layer performs pooling operations:

pooling 3 types

- Max pool
- Average pool
- Min pool

* Convolution and ReLU layers usually stack together after that pooling layer appears

Basic patterns of CNN are shown below.

- Convolution ReLU Convolution ReLU Pooling

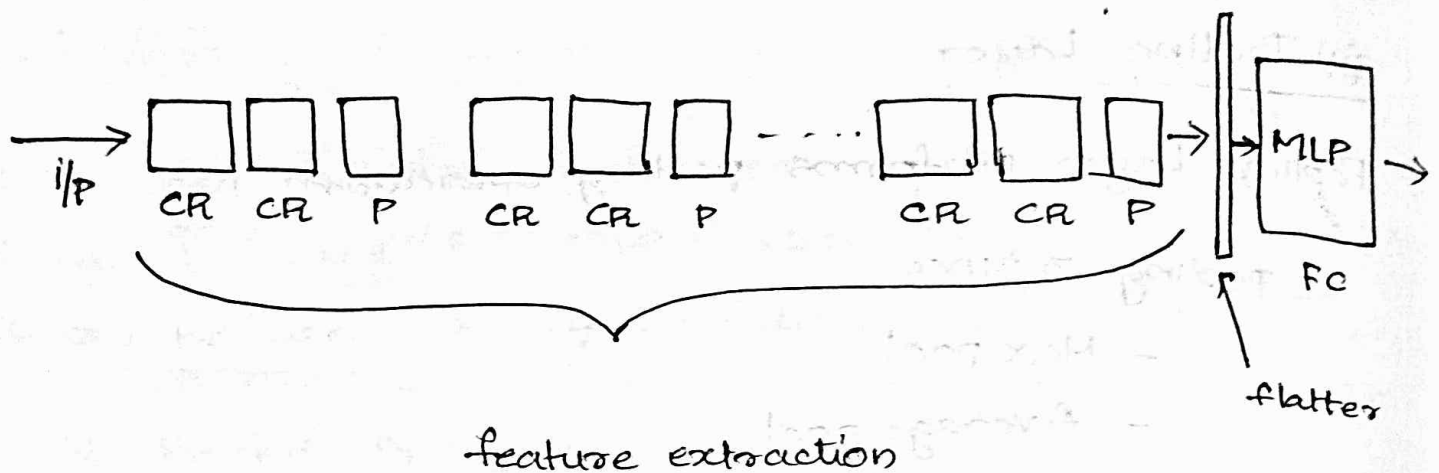


* To create a deep neural network that performs a classification task the above pattern is repeated and at the end a fully connected layer FC is attached.

example



An FC layer is specifically designed to solve perform classification task



CONVOLUTION

Convolution is a linear dot product operations. A convolution operation places a filter (kernel) at each possible position of the image and performs a convolution operation on that positions. The result is a feature map.

data

1	2	3	4
5	6	7	8
9	10	0	1
2	1	2	3

4x4

filter

1	0	0
0	1	0
0	0	1

Findout the feature map after the convolution operation

[Assume stride = 1]

1	2	3	4
5	6	7	8
1	1	0	1
2	1	2	3

1	0	0
0	1	0
0	0	1

$$1 \times 1 + 6 \times 1 = 7$$

1	2	3	4
5	6	7	8
1	1	0	1
2	1	2	3

filter

1	0	0
0	1	0
0	0	1

$$2 \times 1 + 7 \times 1 + 1 \times 1$$

$$= 2 + 7 + 1$$

$$= 10$$

1	2	3	4
5	6	7	8
1	1	0	1
2	1	2	3

1	0	0
0	1	0
0	0	1

$$5 \times 1 + 1 \times 1 + 2 \times 1$$

$$= 5 + 1 + 2$$

$$= 8$$

1	2	3	4
5	6	7	8
1	1	0	1
2	1	2	3

1	0	0
0	1	0
0	0	1

$$6 \times 1 + 0 \times 1 + 3 \times 1$$

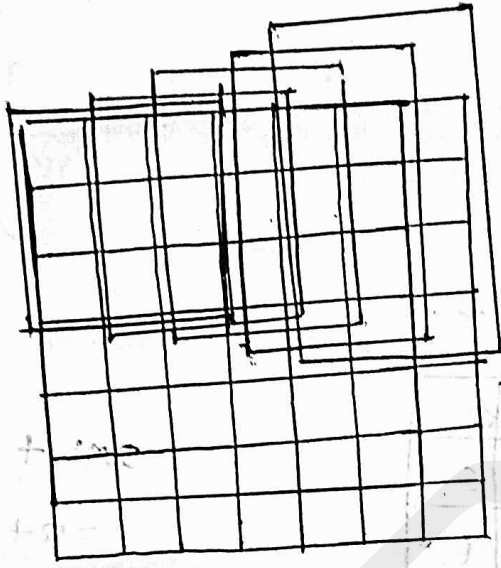
$$= 6 + 0 + 3$$

$$= \underline{\underline{9}}$$



7	10
8	9

Q) How many spatial alignments possible, if the input size is $7 \times 7 \times 1$ and filter size is $3 \times 3 \times 1$. With stride = 1

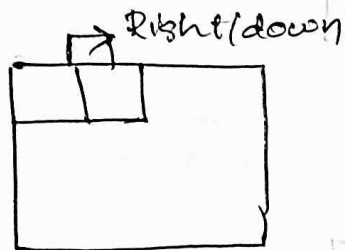


$5 \times 5 \times 1$

Thursday
23/03/2023

Stride

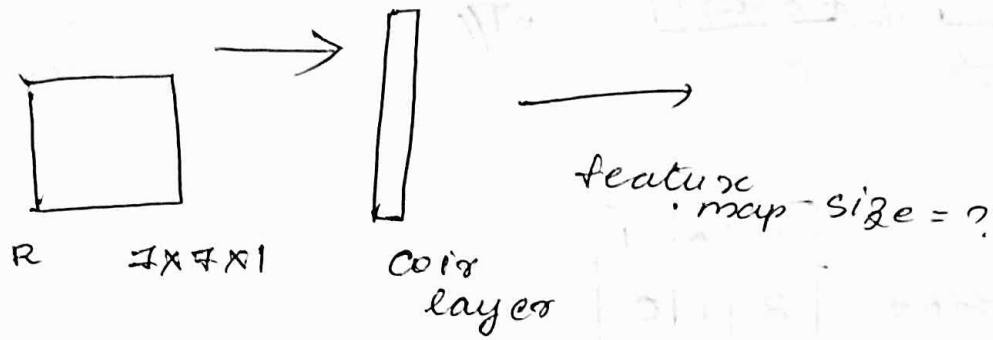
Stride is a parameter that determines the position filter should move.



eg:-

- * When stride = 1, the filter moves one position to the right after performing convolution operation.
- * When stride = 2, the filter moves two position to the right after performing convolution operation.

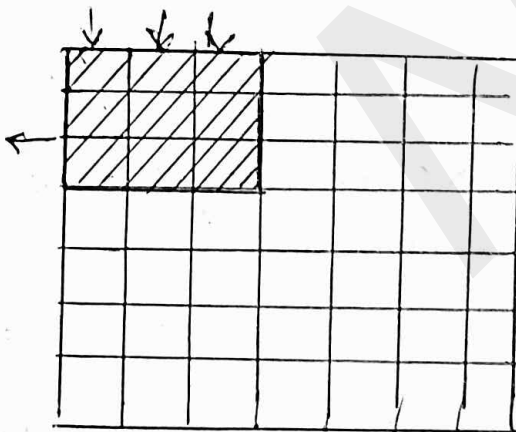
Q) Findout the feature map size for below
 i/p size : $7 \times 7 \times 1$



filter size = $3 \times 3 \times 1$

Stride = 2.

padding = none.



Q) Findout feature map generated

1	1	1	1	1	1	0
0	0	0	1	0	0	0
1	0	0	1	0	0	0
0	1	0	1	0	0	0
1	0	1	0	0	1	0
1	1	0	0	0	0	0
1	1	1	1	1	1	0

filter

1	0	0
0	1	0
0	0	1

3×3

Stride = 2
padding = none

1	1	1
0	0	0
1	0	0

*

1	0	0
0	1	0
0	0	1

$$= 1 \times 1 + 1 \times 0 + 1 \times 0 + 0 \times 0 + 0 \times 1 + 0 \times 0 + 1 \times 0 + 0 \times 0 + 0 \times 1 = 1 //$$

Feature map =

1	2	1
3	1	0
3	2	0

1	1	1
0	0	0
0	0	0

*

1	0	0
0	1	0
0	0	0

=

$$1 \times 1 + 1 \times 0 + 1 \times 0 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 0 \times 0 = 2 //$$

= 2 //

1	1	0
0	0	0
0	0	0

*

1	0	0
0	1	0
0	0	0

=

$$1 \times 1 + 1 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 = 1 //$$

= 1 //

1	0	0
0	1	0
1	0	1

*

1	0	0
0	1	0
0	0	0

=

$$1 \times 1 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 0 \times 0 + 1 \times 1 = 3 //$$

= 3 //

0	1	0
0	1	0
1	0	0

*

1	0	0
0	1	0
0	0	0

=

$$0 \times 1 + 1 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 0 \times 0 + 0 \times 0 = 1 //$$

= 1 //

0	0	0
0	0	0
0	1	0

*

1	0	0
0	1	0
0	0	0

=

$$0 \times 1 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 = 0 //$$

= 0 //

size of the feature map

Let $1/p$ size is $n \times n$, filter size = $f \times f$, stride

= s : then,

$$\begin{aligned} \text{Size of the feature map} &= \\ &= \left(\frac{n-f}{s} + 1 \right) * \left(\frac{n-f}{s} + 1 \right) \end{aligned}$$

Padding

Padding describes the addition of empty pixels all around the ~~nodes~~ borders of the image.

The purpose of padding preserve the original size of the image while doing convolution operations on the image. and perform full convolution on the edge pixels.

eg:-

Consider an image of size 4×4

filter size = 2×2 .

stride = 1

padding = 1

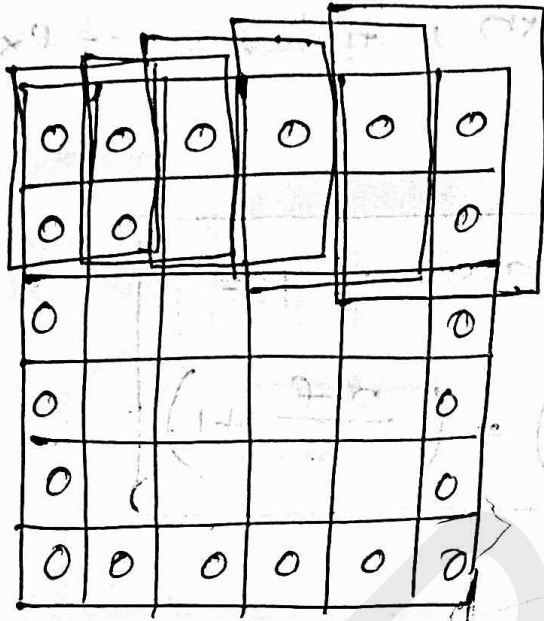
Find out the feature map size.

0	0	0	0	0	0
0					0
0					0
0					0
0					0
0	0	0	0	0	0

padded image

if convolution performed with padding = 1

then feature size = 5×5



In general, while padding $\frac{(f-1)}{2}$ pixels is added all around the borders of an image.

As a result the spatial height and width is increased by $(f-1)$.

* Suppose image size = $n \times n$, feature size = $f \times f$
stride = s , padding = p , Then, the feature map size of feature map generated.

$$= \left(\frac{n + 2p - f}{s} + 1 \right) * \left(\frac{n + 2p - f}{s} + 1 \right)$$

Types of padding

(1) Valid padding

When padding is not used

i) Half padding

$\frac{(F-1)}{2}$ pixels with 0s added all around borders.

0	1	2	3	0
4	5	6	7	8
9	10	11	12	13
14	15	16	17	18
19	20	21	22	23

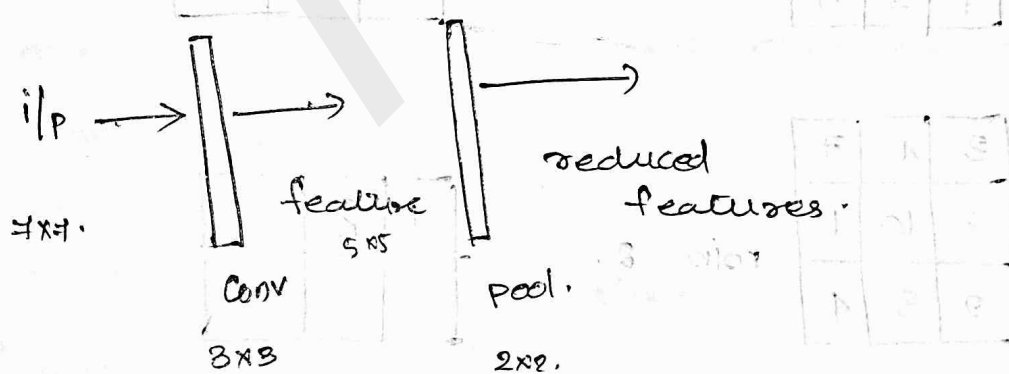
(ii) Full padding

$(F-1)$ pixels w

padding is 'same' if the output feature map is same as the input size

Pooling Layer & Pool operation

Pooling layer is used to reduce the feature map size



Types of pooling

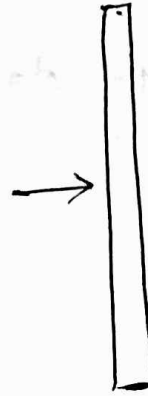
- $\rightarrow$ max pooling
- $\rightarrow$ min pooling
- $\rightarrow$ Average pooling

Example :-

pooling on the feature map

6	3	4	7	8
9	8	10	11	12
1	2	3	4	5
6	7	8	9	10

feature map



pool

Size = 3×3 .

Type = Min Pool

pooling size = 3×3

Type = Min pool.

Stride = 1

6	3	4
9	8	10
1	2	3

min = 1

1		

3	4	7
8	10	11
2	3	4

min = 2

1	2	

4	7	8
10	11	12
3	4	5

min = 3

1	2	3

9	8	10
1	2	3
6	7	8

min = 1

1	2	3
1		

8	10	11
2	3	4
7	8	9

$$\min = 2$$

1	2	3
1	2	0

10	11	12
3	4	5
8	9	10

$$\min = 3$$

1	2	3
1	2	3

* pooling operation works on small grid regions of size $P \times P$ and find maximum or minimum or average values in that grid.

The above process is repeated over the image with stride s .

* Size of the feature map after

$$\text{if } s = 1$$

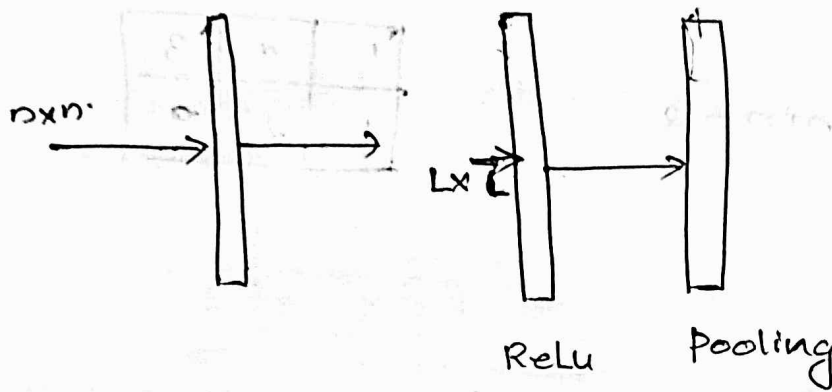
$$(L - P + 1) * (L - P + 1)$$

where,

$$L \rightarrow \text{feature size}$$

$$\text{if } s > 1$$

$$\left(\frac{L - P}{s} + 1 \right) * \left(\frac{L - P}{s} + 1 \right)$$



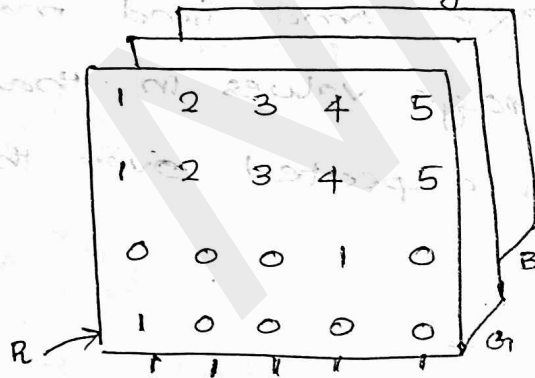
The o/p of convolution layer is feed into a ReLU layer.

Each feature value in the feature map

Tuesday
28/03/2023

Tutorial

Q) Given an RGB image of size $5 \times 5 \times 3$



G

1	0	1	1	0
0	0	1	1	1
1	1	1	1	1
0	0	1	0	1
1	1	0	1	1

B

1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

After

1	0	0
0	1	0
0	0	1

3x3

Find the feature map generated

after RGB is applied to a CNN (Stride = 1)

R channel

1	2	3	4	5
1	2	3	4	5
0	0	0	1	0
1	0	0	0	0
1	1	1	1	1

1	0	0
0	1	0
0	0	1

$$\begin{aligned}
 & 1 \times 1 + 2 \times 1 + 1 \times 0 \\
 & = 1 + 2 \\
 & = 3 //
 \end{aligned}$$

$$2 \times 1 + 3 \times 1 + 1 \times 1 = 2 + 3 + 1 = 6 //$$

$$3 \times 1 + 4 \times 1 + 0 \times 1 = 3 + 4 + 0 = 7 //$$

$$1 \times 1 + 0 \times 1 + 0 \times 0 = 1 //$$

$$2 \times 1 + 0 \times 0 + 0 \times 0 = 2 //$$

$$3 \times 1 + 1 \times 1 + 0 \times 1 = 4 //$$

$$0 \times 1 + 0 \times 1 + 1 \times 1 = 1 //$$

$$0 \times 1 + 0 \times 1 + 1 \times 1 = 1$$

$$0 \times 1 + 0 \times 1 + 1 \times 1 = 1 //$$

3	6	7
1	2	4
1	1	1

Channel

1	0	1	1	0
0	0	1	1	1
1	1	1	1	1
0	0	1	0	1
1	1	0	1	1

1	0	0
0	1	0
0	0	1

$$1 \times 1 + 0 \times 1 + 1 \times 1 = 2 //$$

$$0 \times 1 + 1 \times 1 + 1 \times 1 = 0 + 1 + 1 = 2 //$$

$$1 \times 1 + 1 \times 1 + 1 \times 1 = 1 + 1 + 1 = 3 //$$

$$0 \times 1 + 1 \times 1 + 1 \times 1 = 0 + 1 + 1 = 2 //$$

$$0 \times 1 + 1 \times 1 + 0 \times 1 = 1 //$$

$$1 \times 1 + 1 \times 1 + 1 \times 1 = 1 + 1 + 1 = 3 //$$

$$1 \times 1 + 0 \times 1 + 0 \times 1 = 1 //$$

$$1 \times 1 + 1 \times 1 + 1 \times 1 = 3 //$$

$$1 \times 1 + 0 \times 1 + 1 \times 1 = 1 + 1 = 2 //$$

2	2	3
2	1	3
1	3	2

B channel

1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

1	0	0
0	1	0
0	0	1

$$1 \times 1 + 1 \times 1 + 1 \times 1 = 3$$

$$0 \times 1 + 0 \times 1 + 0 \times 1 = 0$$

$$0 \times 1 + 0 \times 1 + 0 \times 1 = 0$$

$$0 \times 1 + 0 \times 1 + 0 \times 1 = 0$$

$$1 \times 1 + 1 \times 1 + 1 \times 1 = 3$$

$$0 \times 1 + 0 \times 1 + 0 \times 1 = 0$$

$$0 \times 1 + 0 \times 1 + 0 \times 1 = 0$$

$$0 \times 1 + 0 \times 1 + 0 \times 1 = 0$$

$$1 \times 1 + 1 \times 1 + 1 \times 1 = 3$$

3	0	0
0	3	0
0	0	3

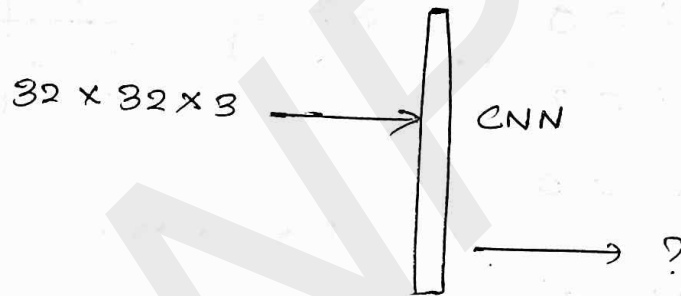
feature map

$3+2+3$	$6+2+0$	$7+3+0$
$1+2+0$	$2+1+3$	$4+3+0$
$1+1+0$	$1+3+0$	$1+2+3$

8	8	10
3	6	7
2	4	6

//

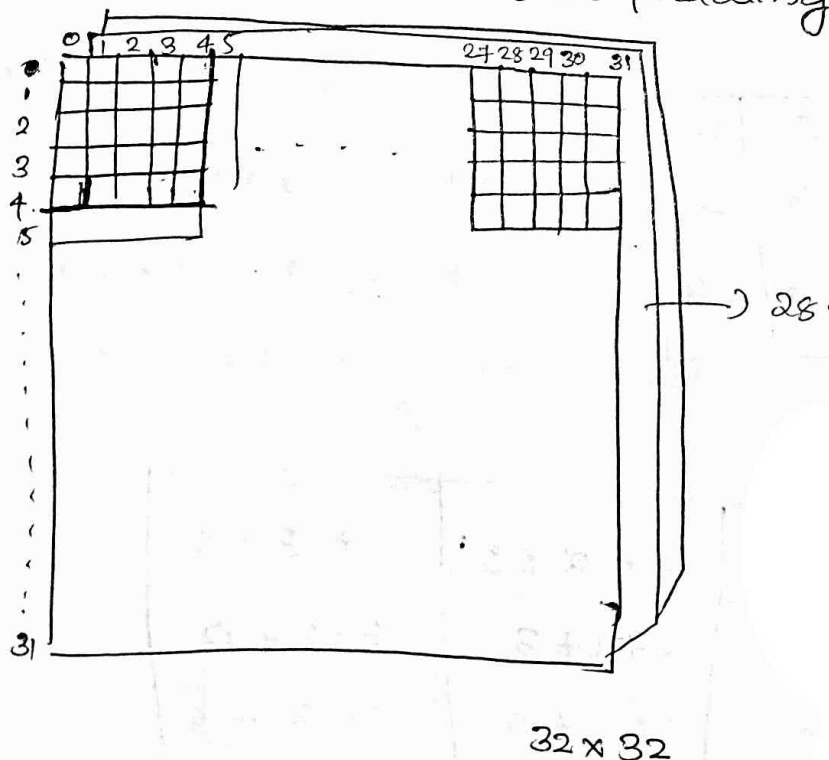
- 2) You have an input volume of size $32 \times 32 \times 3$ what are the dimensions of resulting volume after convolving a $5 \times 5 \times 3$ kernel with zero padding, stride = 2



stride = 2

$5 \times 5 \times 3$

zero padding



general equation

$$\left[\frac{n-f}{s} + 1 \times \frac{n-f}{s} + 1 \right]$$

$s \rightarrow$ stride

$n \rightarrow$

$f \rightarrow$

$$\frac{n-f+1}{s} \times \frac{n-f+1}{s}$$

when $s=1$,

$$\frac{n-f}{s} + 1 \times \frac{n-f}{s} + 1$$

$$= \frac{32-5}{1} + 1 \times \frac{32-5}{1}$$

$$= (32-5+1) \times (32-5+1)$$

$\therefore$ final feature map size

$$\underline{\underline{28 \times 28 \times 1}}$$

when $s=2$.

$$\frac{n-f}{s} + 1 \times \frac{n-f}{s} + 1 = \frac{32-5}{2} + 1 \times \frac{32-5}{2} + 1$$

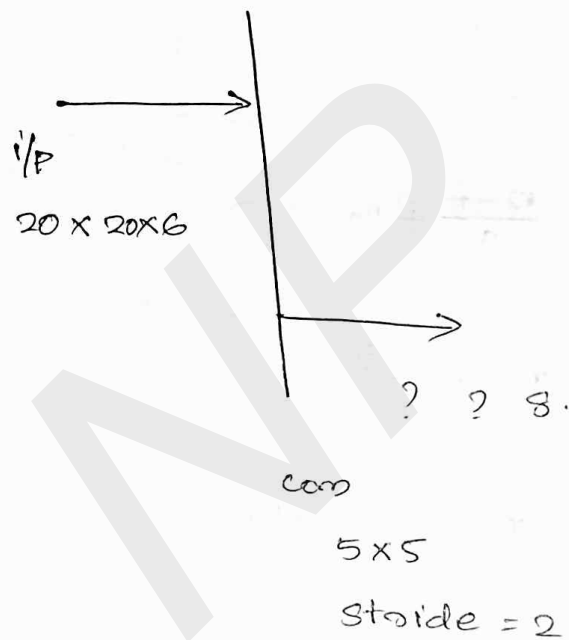
$$= \frac{32-5+1}{2} \times \frac{32-5+1}{2}$$

$$= 14 \times 14$$

$\therefore$ final feature map size

$$14 \times 14 \times 1 //$$

HW Q) consider a convolution layer. The input consists of six feature maps of size 20×20 . The output consists of 8 feature maps and filters are of size 5×5 . The convolution is done with stride of 2 and zero padding. Find out the output feature map size



Thursday
20/09/2023

CNN $\rightarrow$ Sparse Interactions, Parameter sharing, Equivariant representation

* A CNN shows below properties

- ① Sparse interactions
- ② Parameter sharing
- ③ Equivariant Representation.

1. Sparse interactions

- * In a traditional neural network every output unit interact with every input unit

eg:-



- * A CNN has sparse connectivity
- * By using the sparse connectivity concept we can limit the number of connections.

for example :-

Q) consider a $4 \times 4 \times 1$

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

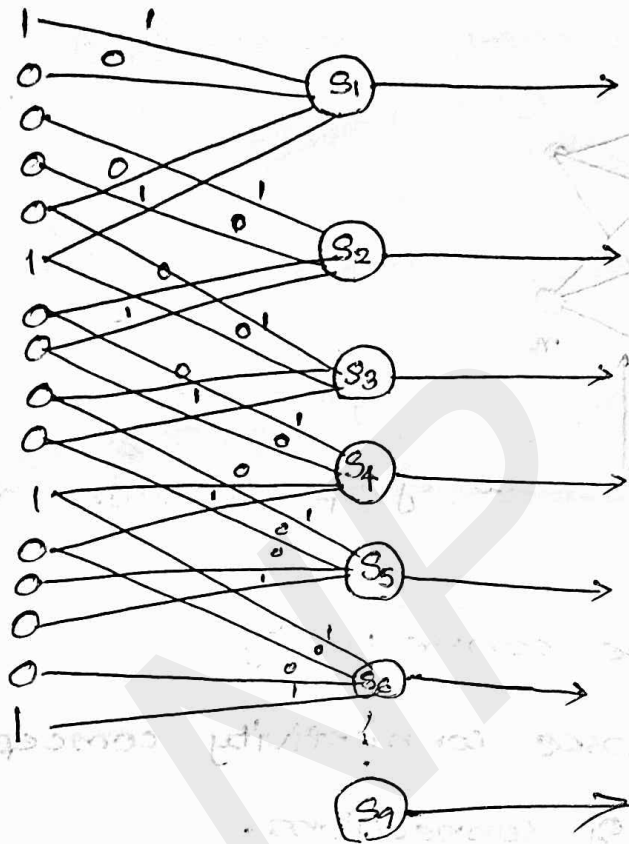
$4 \times 4 \times 1$

and a filter 2×2

1	0
0	1

construct a CNN layer Model using the above information with stride = 1

flattened the matrix



sparse connection

Here,

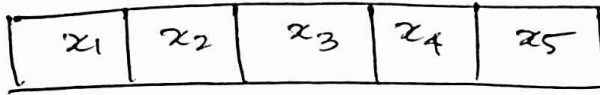
the number of connections

$$= 9 \times 4$$

$$= 36$$

* If we use a traditional Neural network. Then the number of connections needed will be greater than the CNN

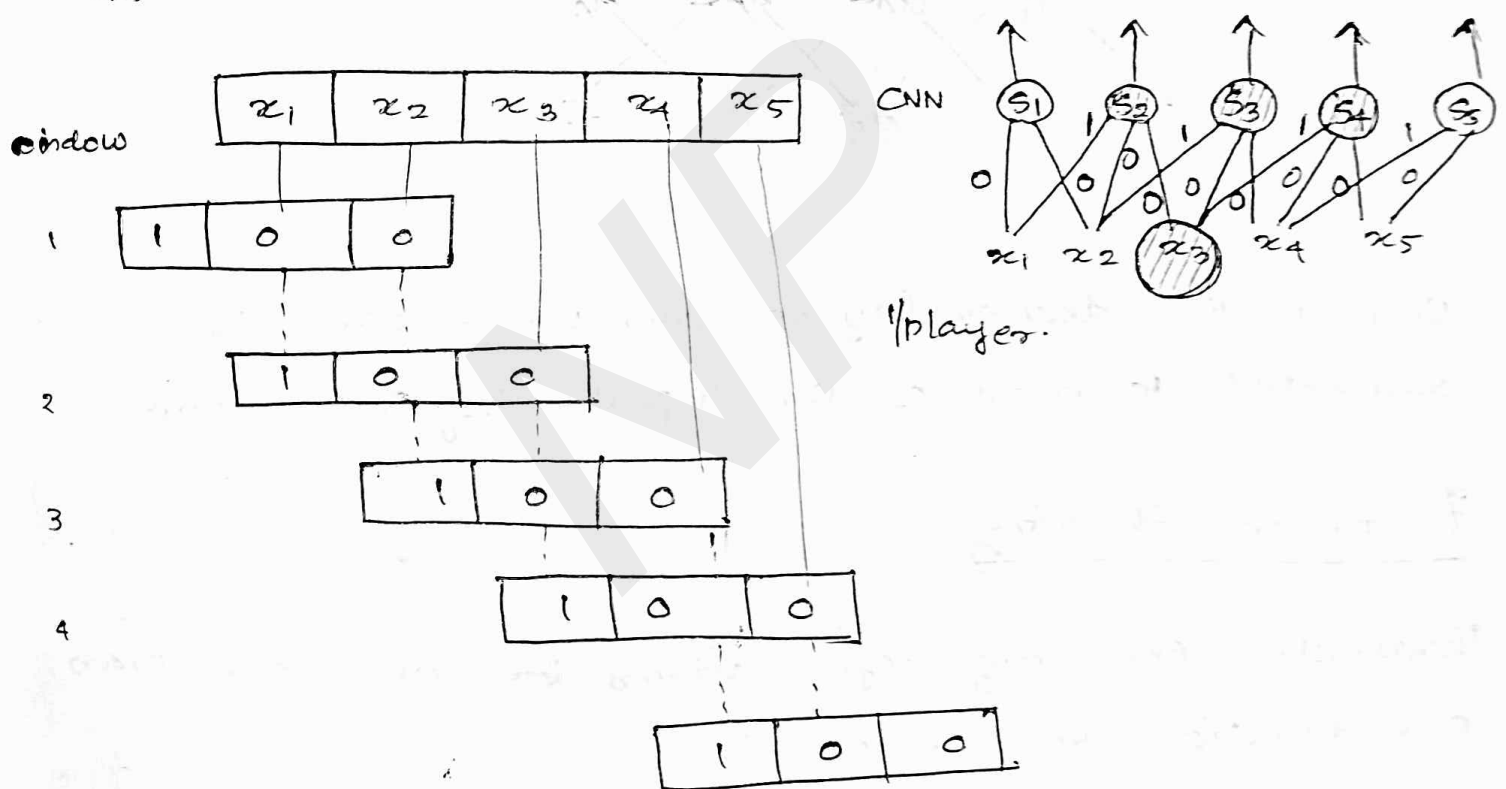
Consider an input array of one dimension



and consider a window of width = 3

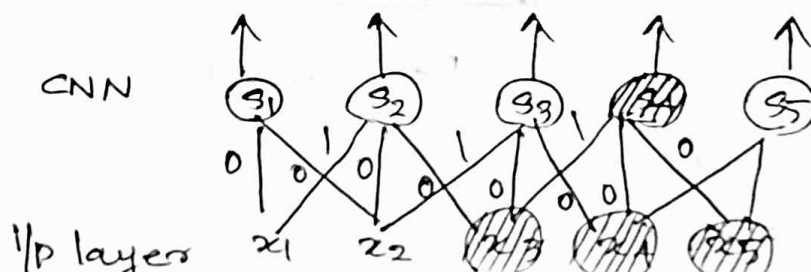
Construct a CNN model of the above configuration with stride = 1

Ans



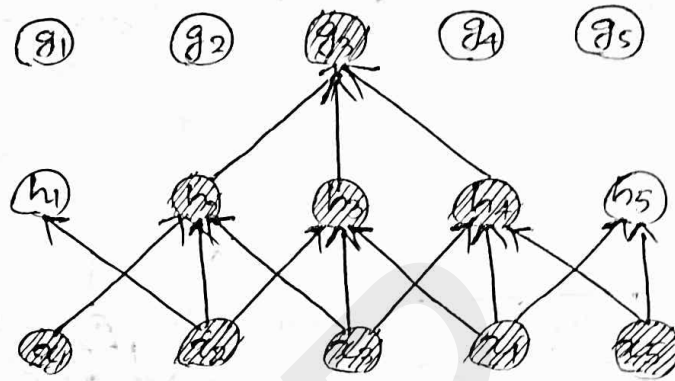
In this case number of connections is number of connections is 13

Receptive field : Receptive field of S_4 is



Receptive fields of the units in the deeper layer of a CNN is larger than receptive field of units in the shallow layers.

eg:-



* Units in the deeper layers can be indirectly connected to most of the input layer's neurons

Parameter Sharing

Parameter sharing refers using the same for more than one function in the model.

eg:-

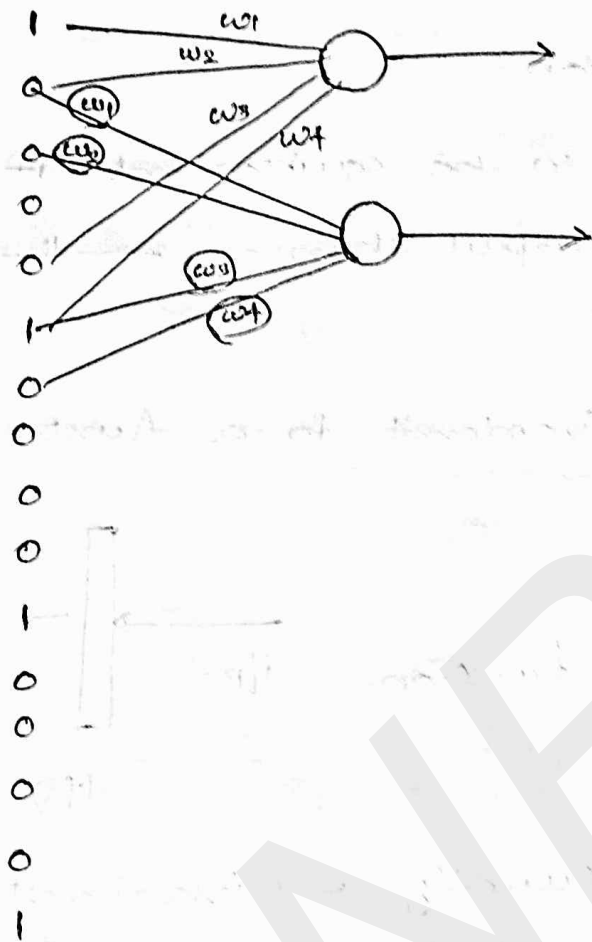
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

w_1	w_2
w_3	w_4

Edge detector

Corresponding CNN model.

(parameter sharing)



- * A CNN automatically find parameters values that learns to perform edge detection etc. in the input image.
- * No separate set of parameters required at different locations to detect edges

CNN compared to Parameters shared along

Neural Network $\Rightarrow 9 \times 4 = 36$

ADVANTAGES

- * A CNN reduces the memory requirements and statistical efficiency.

3. Equivariance

Equivariant Representation.

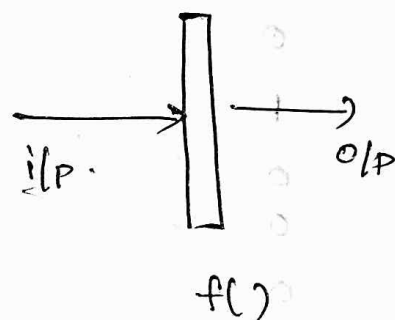
A function is said to be equivariant if the input changes then the output changes ~~in the~~.

A CNN is equivariant

A function $f(x)$ is equivariant to a function g

if
$$f(g(x)) = g(f(x))$$

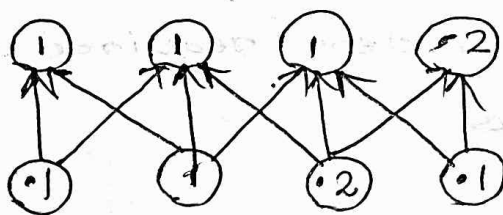
$g \rightarrow$ transformation function
(shift function)



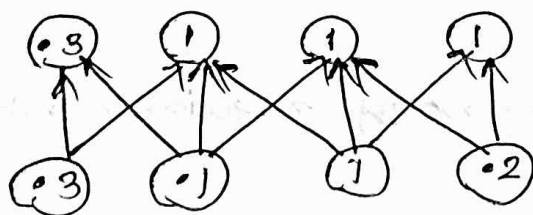
* Convolution is not naturally equivariant to other transformations such as
 $\rightarrow$ change in Scale rotation.

Pooling and Invariance

Pool:



Pool:

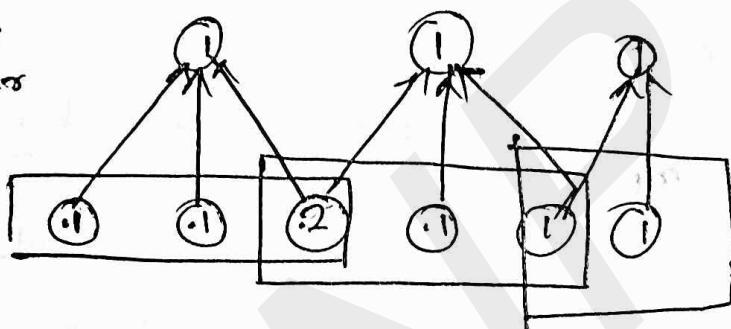


In the bottom figure input has been shifted to right by one pixel.

Every value in the input is changed after the shift but only half of the values in the pooling layer is changed.

Pooling and down Sampling

max
pool
layer



Window Size = 3
Stride = 2

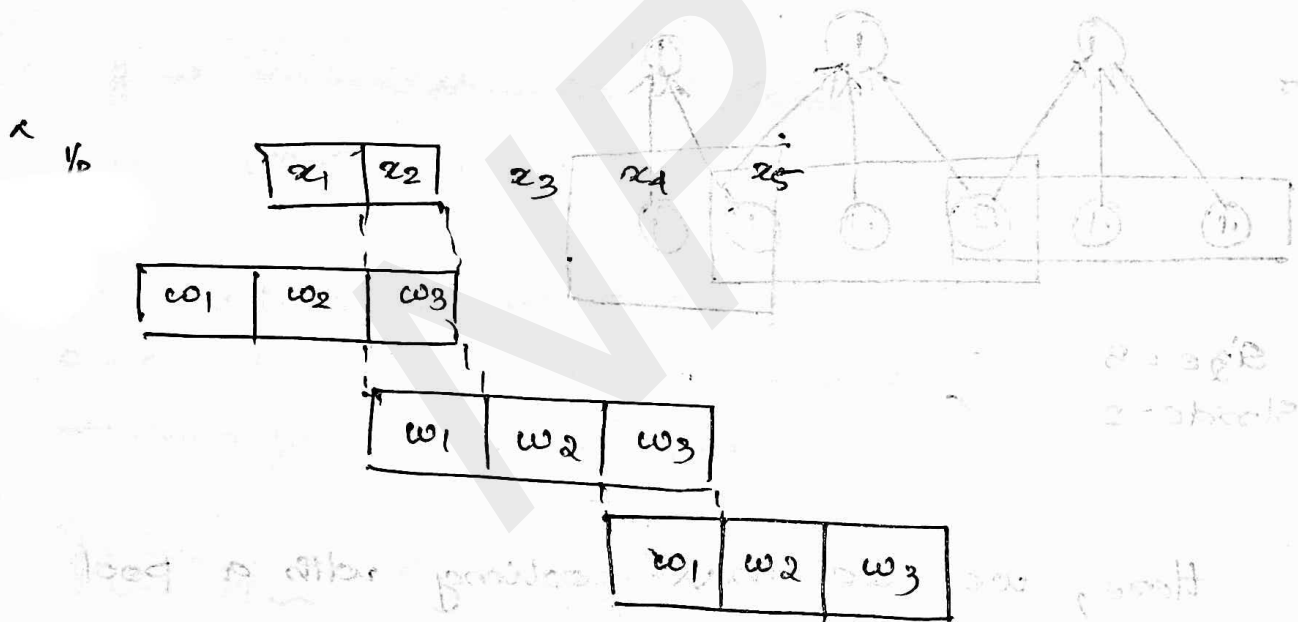
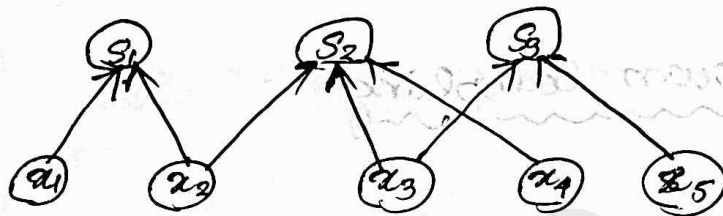
Here, we use 'max' pooling with a pool width of 3 and the stride between the pools equal to 2. This reduces the representation size by the factor of 2.

The pooling operation reduces the computational cost by reducing the number of parameters to the next layer.

Variants of basic convolution

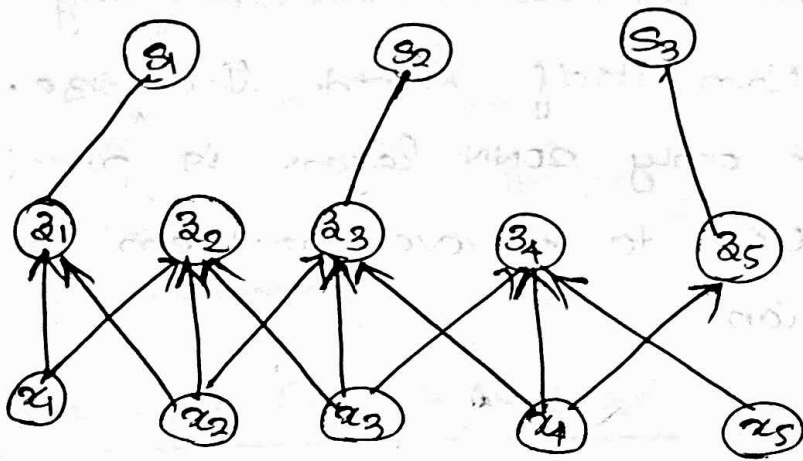
1. Strided convolution

It is possible to use strides of different sizes to perform convolution for example stride = 2, 3 etc.



* Convolution with a stride greater than one pixel mathematically is equivalent to convolution with unit stride followed by down sampling.

* But the two step approach involving unit stride followed by down sampling is computationally wasteful.



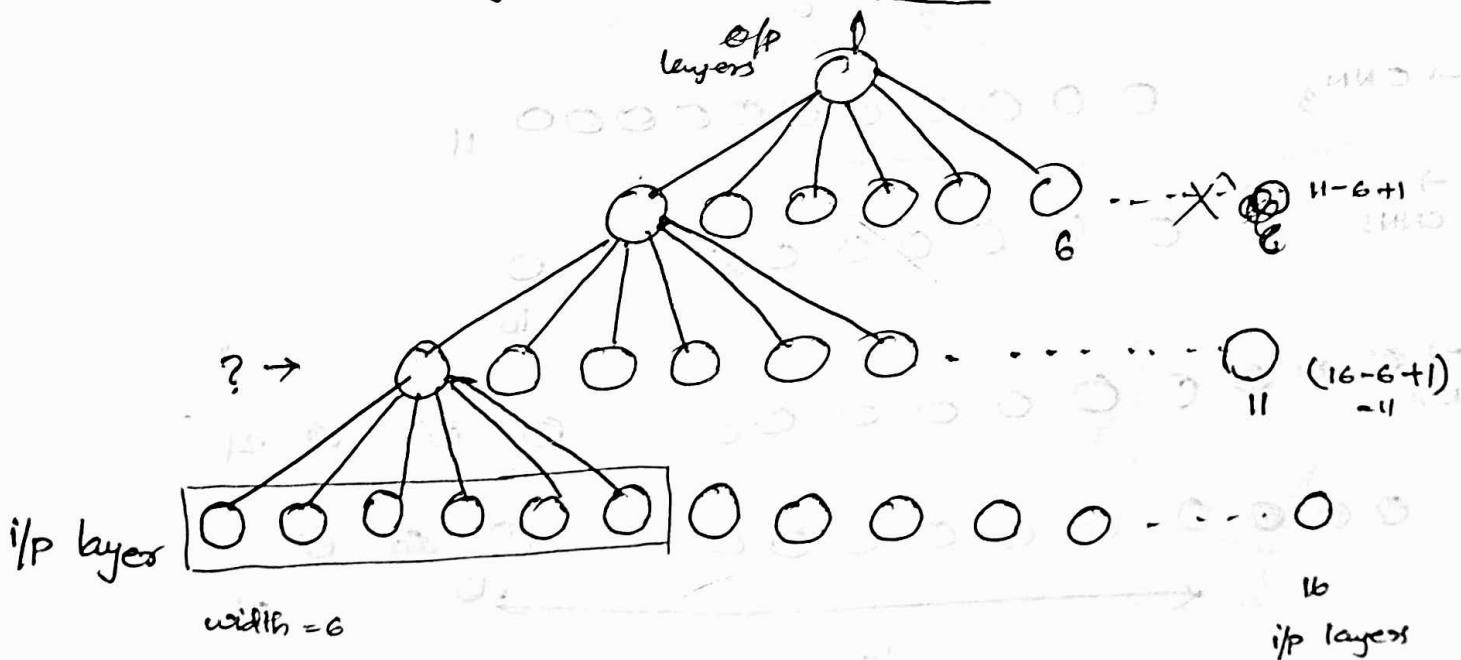
Tuesday
4/04/2023

Padded convolution

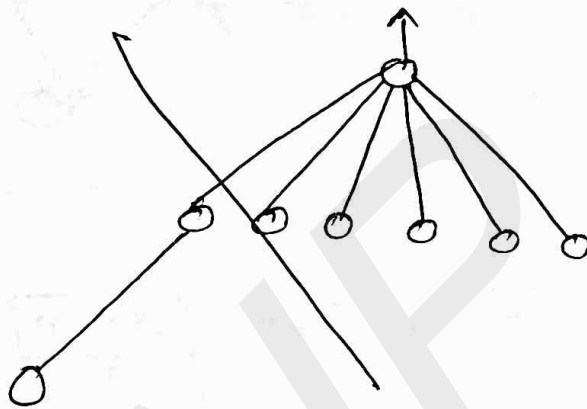
Types

- Valid (no padding)
- Half
- Full
- Same

Effect of Padding on Network size

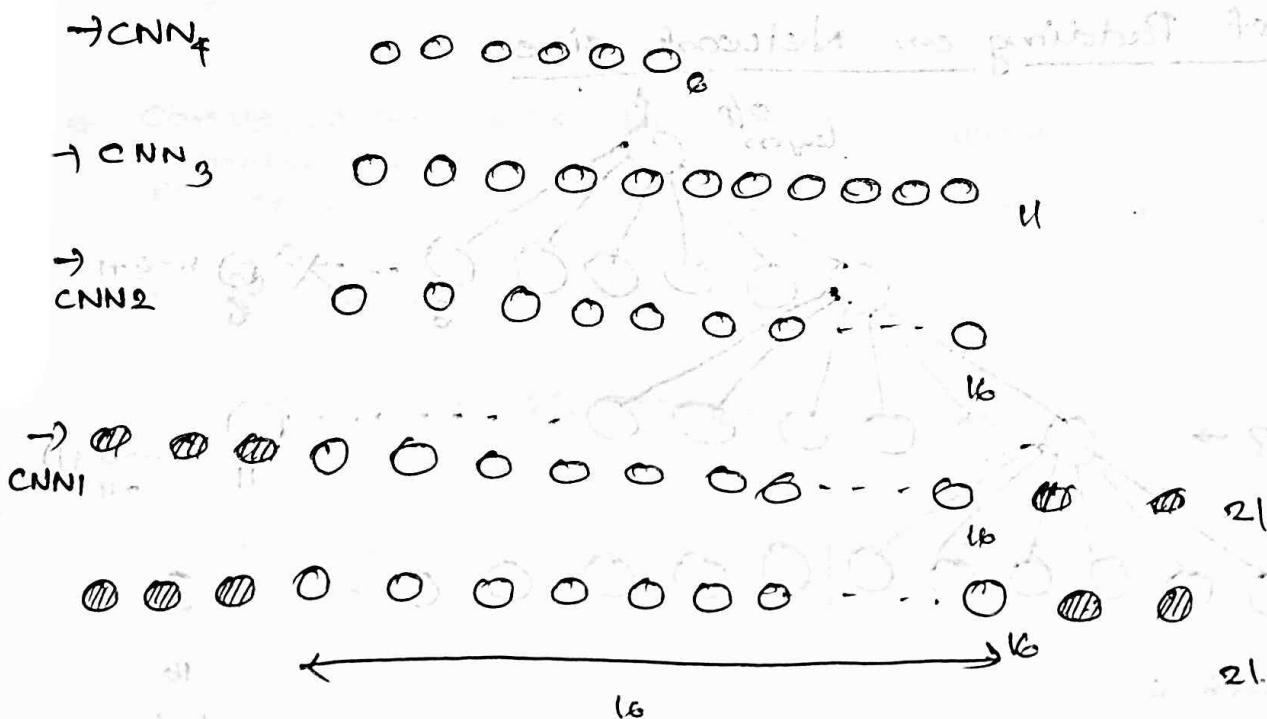


In this example we do not use pooling the ^{n/w} convolution operation itself shrink the size, by 5 pixels at each layer so only 2 CNN layers is possible in this case solution to above problem is to use padded convolution.



padding convolution

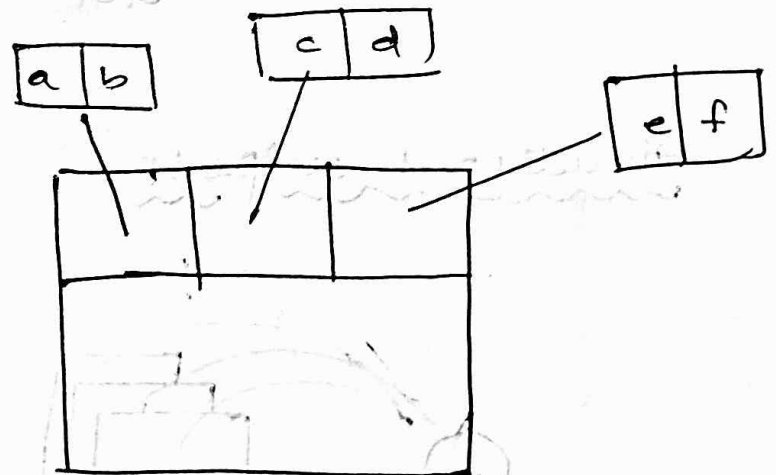
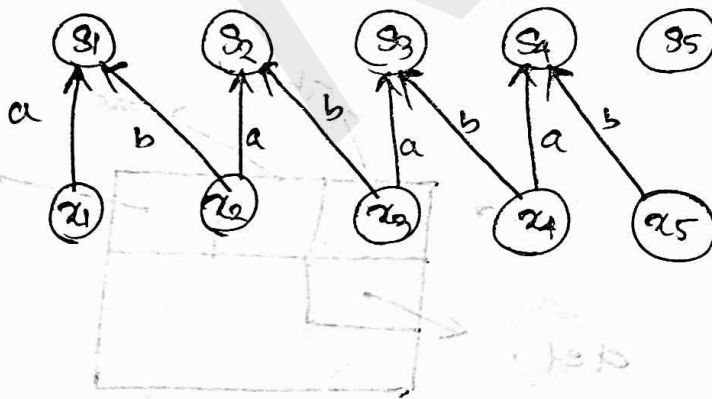
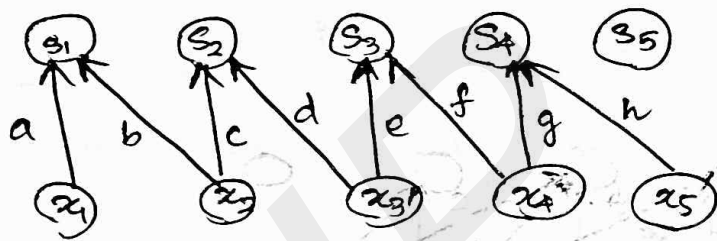
considering an input $\leftarrow$
 filter $\leftarrow$
 shift $\leftarrow$
 sum $\leftarrow$



After performing padding on each layer the number of CNN layers that can be added to the network is increased.

~~unshared~~

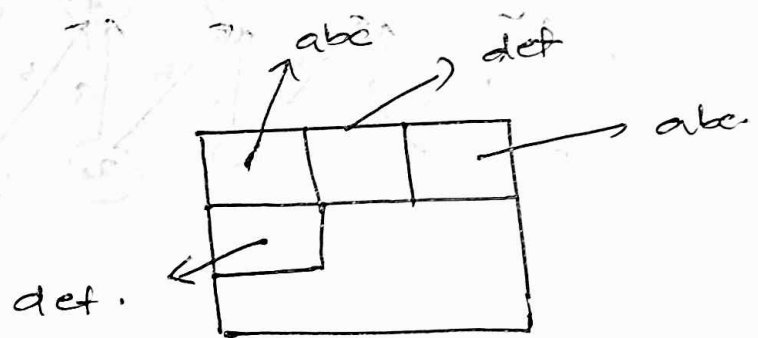
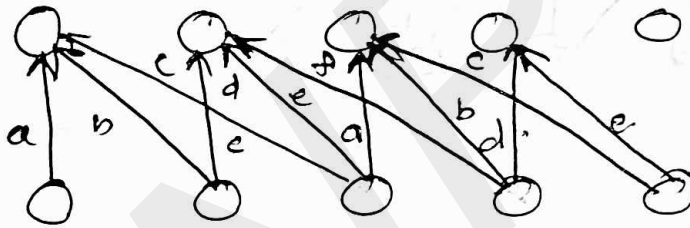
Unshared Convolution



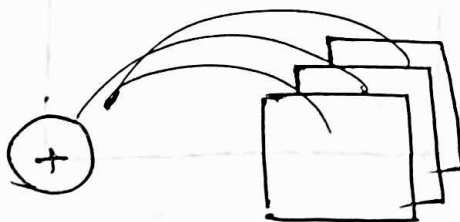
In an unshared convolution different parameters are used at each convolution.

Structured Outputs

Tiled Convolution



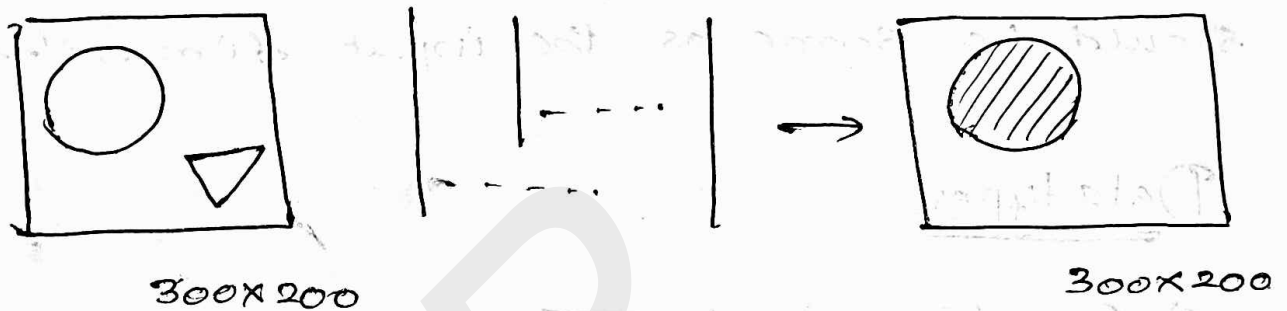
Structured outputs



convolution neural network can be used to output a high dimensional structured objects rather than just predicting the class label for a classification task or a real value for a regression task.

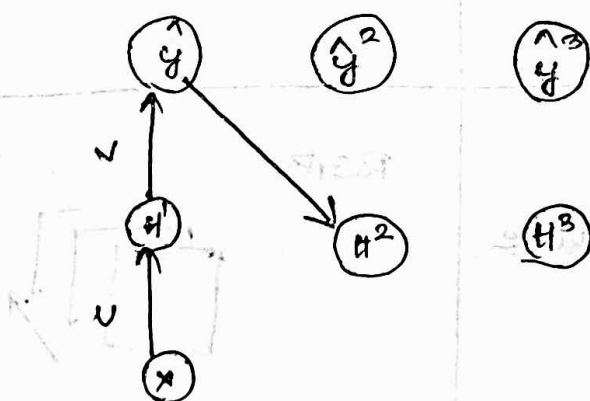
Segmentation work

eg:-



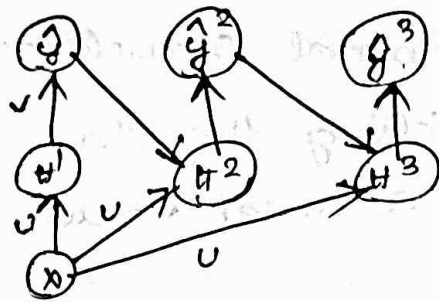
In such task the model needs to label every pixel in an image and draw precise the mask that follow the outline of individual objects

Example: — Architecture.



$x \rightarrow$ $1/p$ pixel

$y \rightarrow$ corresponding label



Process repeat until
reaches the correct label.

One issue need to handle is output dimension
should be same as the input dimensions.

Datatypes

a) One dimensional data.

Single channel

Multi channel

ONE DIMENSIONAL DATA.

a) Audio data



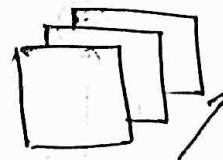
a) Animation data

TWO DIMENSIONAL DATA.

Grey scale/ Black and white



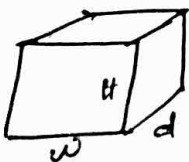
RGB



THREE DIMENSIONAL DATA.

Volumetric data

eg:- CT scan image
etc



Colour
Video data

